

Evidentia Project Setup & Configuration Documentation

DEPLOYMENT REFERENCE • DATE: JUNE 19, 2026

1. Additional Dependencies Installed

The following core packages have been explicitly installed via npm inside the `frontend/` directory to manage stateful routing, HTTP clients, and modern style processing utilities:

```
npm install axios
npm install react-router-dom
npm install -D tailwindcss postcss autoprefixer @tailwindcss/postcss
```

2. Database Structure (MySQL System)

As verified via the administration environment (phpMyAdmin), the application's users table structure is configured as follows. Note that the table utilizes a `password_hash` field alongside an explicit schema architecture tracking precise clearance roles.

Table Schema

```
CREATE TABLE users (
  id INT AUTO_INCREMENT PRIMARY KEY,
  username VARCHAR(50) NOT NULL UNIQUE,
  password_hash VARCHAR(255) NOT NULL,
  role ENUM('admin', 'officer') NOT NULL,
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;
```

Active Test Users Seed Data

id	username	password_hash	role
1	admin	admin	admin
2	officer	officer	officer

Security Warning: The current development environment stores passwords in plain-text inside the `password_hash` column. This approach is designated strictly for active test scenarios and local prototyping. It

must be updated to robust one-way cryptographic hashing (e.g., `password_hash()` with `PASSWORD_BCRYPT`) prior to production staging.

3. Database Connection Configuration

File Path: `backend/config/db_connect.php`

```
<?php
header("Access-Control-Allow-Origin: *");
header("Access-Control-Allow-Methods: POST, GET, OPTIONS, DELETE");
header("Access-Control-Allow-Headers: Content-Type, Authorization, X-Requested-With,
ngrok-skip-browser-warning");
header("Content-Type: application/json");

if ($_SERVER['REQUEST_METHOD'] == 'OPTIONS') {
    http_response_code(200);
    exit();
}

$host = "localhost";
$dbname = "evidentia_db";
$username = "root";
$password = "amNekku2026";

try {
    $pdo = new PDO("mysql:host=$host;dbname=$dbname;charset=utf8mb4", $username,
$password);
    $pdo->setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION);
    $pdo->setAttribute(PDO::ATTR_DEFAULT_FETCH_MODE, PDO::FETCH_ASSOC);
} catch (PDOException $e) {
    http_response_code(500);
    echo json_encode([
        "status" => "error",
        "message" => "Avena: Database connection failed",
        "debug" => $e->getMessage()
    ]);
    exit();
}
?>
```

4. Authentication API Endpoint

File Path: `backend/api/login.php`

```
<?php
require_once __DIR__ . '/../config/db_connect.php';
```

```

if ($_SERVER['REQUEST_METHOD'] !== 'POST') {
    http_response_code(405);
    echo json_encode(["status" => "error", "message" => "Method Not Allowed"]);
    exit();
}

$inputData = json_decode(file_get_contents("php://input"), true);
$username = trim($inputData['username'] ?? '');
$password = trim($inputData['password'] ?? '');

if (empty($username) || empty($password)) {
    http_response_code(400);
    echo json_encode(["status" => "error", "message" => "Please fill out all fields."]);
    exit();
}

try {
    $stmt = $pdo->prepare("SELECT id, username, password_hash, role FROM users WHERE
username = :username LIMIT 1");
    $stmt->execute(['username' => $username]);
    $user = $stmt->fetch();

    if ($user && $password === $user['password_hash']) {
        echo json_encode([
            "status" => "success",
            "message" => "Login successful!",
            "user" => [
                "id" => $user['id'],
                "username" => $user['username'],
                "role" => $user['role']
            ]
        ]);
    } else {
        http_response_code(401);
        echo json_encode(["status" => "error", "message" => "Invalid username or
password."]);
    }
} catch (PDOException $e) {
    http_response_code(500);
    echo json_encode(["status" => "error", "message" => "Server transaction failure."]);
    exit();
}
?>

```

5. Frontend UI Implementation

File Path: frontend/src/App.jsx

```
// eslint-disable-next-line no-unused-vars
import React, { useState } from 'react';
import './index.css'

export default function App() {
  const [form, setForm] = useState({ username: '', password: '' });
  const [error, setError] = useState('');
  const [success, setSuccess] = useState('');

  const handleChange = (e) => {
    setForm({ ...form, [e.target.name]: e.target.value });
  };

  const handleLogin = async (e) => {
    e.preventDefault();
    setError('');
    setSuccess('');

    try {
      const response = await fetch('http://localhost:8080/login.php', {
        method: 'POST',
        headers: { 'Content-Type': 'application/json' },
        body: JSON.stringify(form),
      });

      const data = await response.json();

      if (!response.ok) {
        throw new Error(data.message || 'Authentication failed');
      }

      setSuccess(`Access Granted! Logged in as ${data.user.role.toUpperCase()}.`);
    } catch (err) {
      setError(err.message);
    }
  };

  return (
    <div className="min-h-screen flex items-center justify-center bg-slate-900 px-4">
      <div className="max-w-md w-full bg-slate-800 p-8 rounded-xl shadow-xl border border-slate-700">
        <div className="text-center mb-8">
          <h1 className="text-3xl font-bold text-white tracking-tight">Evidentia</h1>
          <p className="text-slate-400 mt-2 text-sm">Account Portal Access</p>
        </div>
      </div>
    </div>
  );
}
```

```

    </div>

    {error && <div className="mb-4 p-3 bg-red-500/10 border border-red-500/30 text-
red-400 text-sm rounded-lg">{error}</div>}
    {success && <div className="mb-4 p-3 bg-emerald-500/10 border border-
emerald-500/30 text-emerald-400 text-sm rounded-lg">{success}</div>}

    <form onSubmit={handleLogin} className="space-y-5">
      <div>
        <label className="block text-sm font-medium text-slate-300 mb-1.5">Username</
label>
        <input
          type="text"
          name="username"
          value={form.username}
          onChange={handleChange}
          className="w-full px-4 py-2.5 bg-slate-900 border border-slate-700 rounded-
lg text-white focus:outline-none focus:border-sky-500 transition-colors"
          placeholder="Username"
          required
        />
      </div>

      <div>
        <label className="block text-sm font-medium text-slate-300 mb-1.5">Password</
label>
        <input
          type="password"
          name="password"
          value={form.password}
          onChange={handleChange}
          className="w-full px-4 py-2.5 bg-slate-900 border border-slate-700 rounded-
lg text-white focus:outline-none focus:border-sky-500 transition-colors"
          placeholder="....."
          required
        />
      </div>

      <button type="submit" className="w-full py-3 px-4 bg-sky-600 hover:bg-sky-500
text-white font-medium rounded-lg shadow transition-colors duration-200 mt-2">
        Sign In
      </button>
    </form>
  </div>
</div>
);
}

```

6. Project Local Execution & Administration

- **Backend API Server (PHP):** Runs on `http://localhost:8080` to bypass/avoid routing conflicts with any active local DSpace application instances.
- **Frontend Development Server (Vite):** Accessible locally via its default development port (e.g., `http://localhost:5173` or configured alternative).

Administrative Routine & Database Commands

To safely spin up the database user interface and verify the service statuses, utilize the terminal shortcuts outlined below:

Serve phpMyAdmin Interface locally:

```
cd /usr/share/phpmyadmin && sudo php -S localhost:8080
```

Initialize & verify regional MySQL Service Status:

```
cd Documents/messyMesh && sudo systemctl start mysql
```